

INDIAN INSTITUTE OF INFORMATION
TECHNOLOGY DESIGN AND MANUFACTURING
KANCHEEPURAM
CHENNAI-600127, TAMILNADU



Smart Product Design

Opticane

Team - 17

Introduction:

OptiCane is a smart, portable assistive device designed to enhance the safety and independence of visually impaired individuals. Developed as an add-on to a traditional walking cane, it integrates two essential features—fall detection and an SOS emergency alert system—to provide an added layer of security during daily movement. With its compact and detachable design, OptiCane aims to be a reliable and user-friendly solution that empowers users to navigate with greater confidence.

Motivation:

Visually impaired individuals face significant challenges in their daily lives, especially when it comes to mobility and ensuring personal safety. Traditional white canes, while helpful for navigation, offer no support in emergency situations such as accidental falls or sudden health issues. In many cases, a delay in receiving help during such incidents can lead to serious consequences.

Recognizing these challenges, we were motivated to develop OptiCane—a smart, portable device that adds essential safety features like fall detection and an SOS alert system to a standard cane. Our goal is to provide a reliable assistive tool that empowers visually impaired users with a greater sense of independence, security, and peace of mind. By integrating simple yet effective technology, OptiCane bridges a critical gap between basic mobility aids and modern smart assistance, without compromising on usability or comfort.

Objectives:

- To develop a portable assistive device that can be attached to a regular walking cane used by visually impaired individuals.
- To implement a reliable fall detection system using motion sensors that can automatically identify accidental falls.
- To integrate a manual SOS alert feature allowing users to seek immediate help in emergencies.
- To ensure the system is compact, power-efficient, and easy to use, without disrupting the primary function of the cane.
- To provide prototype-level validation of safety functionalities through hardware integration and testing.

Scope of Work:

- The project focuses on designing and developing a prototype that includes:
 1. A motion detection module for fall recognition.
 2. An emergency alert system using a push-button and communication module.
 3. A safe button mechanism for false alarm cancellation.
- The current scope does not include navigation or obstacle detection functionalities
- The design is limited to core safety features, aiming for user-friendliness, minimal hardware complexity, and portability.
- The prototype will be tested under controlled conditions to demonstrate its functionality and reliability.

With a clear focus on enhancing user safety through essential features, it is important to review existing solutions and research in related domains.

Literature Survey:

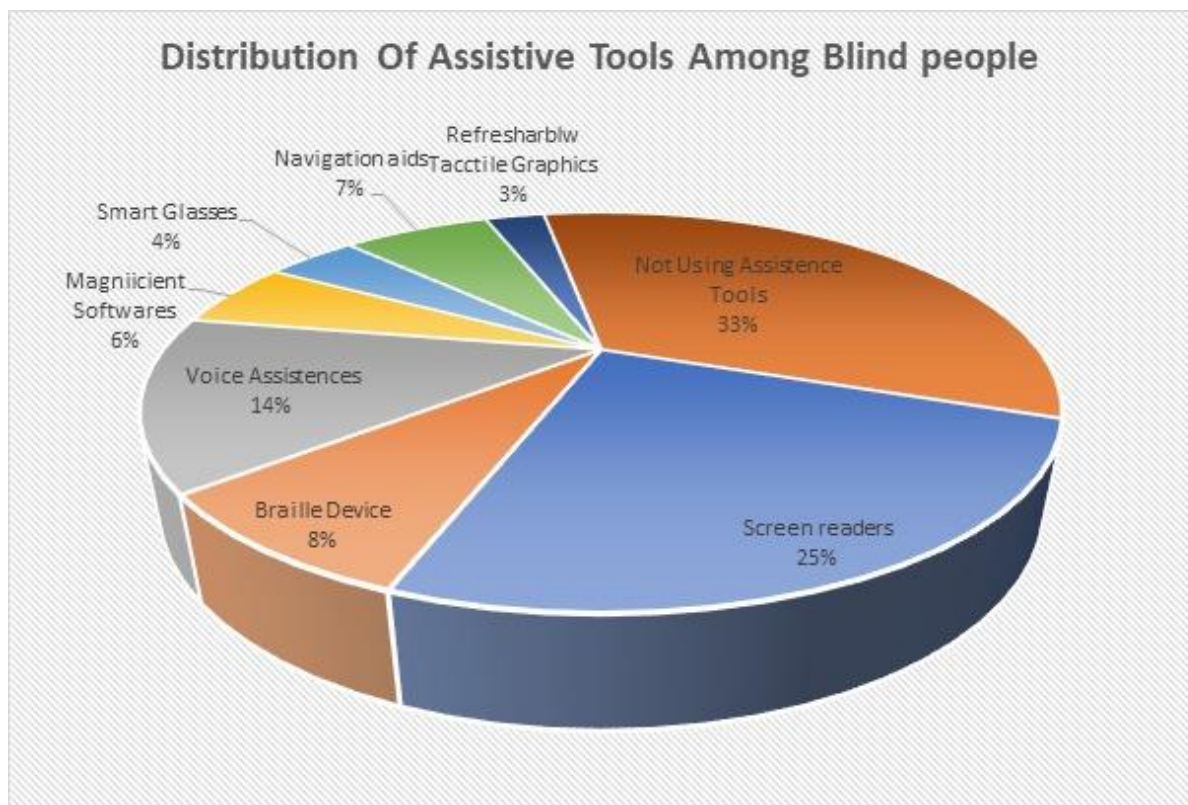
Over the years, numerous research efforts and innovations have been directed toward assistive technologies for the visually impaired, particularly focusing on navigation and safety. While many devices enhance mobility through obstacle detection using ultrasonic or infrared sensors, fewer solutions emphasize emergency response mechanisms such as fall detection or SOS alerts.

Several studies have explored the use of accelerometer and gyroscope data for detecting unusual motion patterns that may indicate a fall. For instance, the MPU-6050 sensor, combining both accelerometer and gyroscope functionalities, has been widely used in wearable and portable devices to identify sudden changes in motion, which can be interpreted as falls. These solutions often employ threshold-based algorithms or machine learning techniques to distinguish between normal activity and critical events.

In terms of communication, systems such as GSM modules and microcontrollers like the ESP32 or Arduino have been used to send emergency alerts. These alerts often include real-time GPS location tracking to inform caregivers or emergency responders, a concept that has been applied in elder-care and health-monitoring systems.

However, most of these solutions are bulky, expensive, or not tailored for visually impaired users. Few integrate both fall detection and manual SOS alerts into a compact, attachable form factor for walking canes. This gap in user-focused safety solutions highlights the need for a device like OptiCane, which prioritizes simplicity, portability, and reliability.

OptiCane draws inspiration from these existing studies and devices, while narrowing its focus to provide core safety features in a form that complements traditional mobility aids.



Methodology:

The development process of *Opticane* was carried out in carefully planned phases to ensure accurate implementation of its core features—**Fall Detection** and **SOS Emergency Alert**. The methodology adopted is detailed below:

1. Problem Identification:

The first step was to clearly define the problem faced by visually impaired individuals in everyday scenarios, especially in terms of personal safety. The inability to promptly call for help during a fall or emergency was identified as a critical issue. This guided our project toward integrating an intelligent alert mechanism that would function reliably in real-time. The goal was to enable autonomous safety intervention without requiring continuous user input.

2. Feature Selection and Scope Definition:

Following the problem analysis, various possible features were brainstormed. For the initial prototype, it was decided to focus solely on Fall Detection and SOS Alert, as they were both highly impactful and feasible to implement within the project timeline. This narrowed scope allowed the team to concentrate efforts on perfecting these two essential functions before expanding to more advanced features in the future. The selected features directly addressed the need for immediate support during critical situations and offered both automatic and manual activation modes.

3. Planning the Approach:

To bring the selected features to life, a structured plan was formed that focused on system behaviour rather than individual hardware components. The approach for fall detection involved monitoring motion patterns and sudden changes in orientation to recognize when a user has potentially fallen. A delay mechanism was integrated to minimize false alarms by giving the user a window to cancel the alert if no help is needed. Parallely, a manual SOS system was planned, which would allow users to instantly send a distress signal by pressing a dedicated button. This dual-mode emergency system was designed to ensure both responsiveness and user control.

4. Software Logic and Workflow:

After establishing the overall system behaviour, software development was initiated to implement the logic for real-time data processing and alert triggering. Sensor data would be continuously monitored, and specific thresholds were defined to identify fall-like events. Once detected, the system would enter a short delay period, allowing the user to stop the SOS trigger manually in case of a false positive. Simultaneously, the SOS button was programmed to override this logic and immediately activate alerts regardless of sensor readings. The software was designed to be robust and adaptable for future feature additions.

5. Testing and Iteration:

The final phase involved rigorous testing to ensure the reliability and accuracy of the implemented features. Various test scenarios were created, such as walking, sudden sitting, dropping the cane, and actual fall simulations to verify the effectiveness of the fall detection algorithm. The timing and reliability of the SOS trigger, both manual and automatic, were closely observed. Based on these tests, parameters like sensitivity thresholds and delay durations were fine-tuned to strike the right balance between responsiveness and error prevention. Iterative refinements were carried out to improve overall system performance and user experience.

6. 3D Model Planning:

In parallel with the system development, a physical housing design was conceptualized to integrate all components into a compact, detachable unit. The design goal was to maintain the cane's ergonomics and usability while securely enclosing the electronics. CAD software was used to model the enclosure, keeping in mind accessibility to the SOS button and visibility for indicators. Special attention was given to ensuring that the model could fit onto any standard walking cane, thereby making the device modular and universally adaptable. This planned enclosure forms an essential part of the final product's user-friendliness and portability.

Components:

The design of the Opticane smart cane relies on a careful selection of electronic components, each playing a specific and essential role in the functionality of the device. These components work together to ensure accurate fall detection, reliable SOS alerting, and overall system stability. Below is a description of each component and its role in the prototype:

The **ESP32 microcontroller** serves as the brain of the system. It processes sensor data, handles button inputs, and controls the logic for fall detection and SOS alerts. Its built-in Wi-Fi and Bluetooth capabilities also make it future-ready for wireless alert systems.

The **MPU-6050 sensor module**, which combines a 3-axis accelerometer and a 3-axis gyroscope, is used to detect motion and orientation changes. It is responsible for identifying sudden falls by analysing variations in acceleration and angular velocity.

The **NEO-6M GPS module** provides real-time location tracking. Although not fully integrated in this prototype, it supports future scalability where SOS alerts could include GPS coordinates for enhanced user safety.

To ensure smooth power management and reliable user interaction, several supporting components were integrated into the system. Two **self-reset push buttons** are used—one to manually trigger an SOS alert and the other to cancel a fall-based alert in case of a false detection, giving users both autonomy and safety. A **slide switch** is included to allow the user to turn the device on or off as needed, helping conserve battery when not in use. The system is powered by a **rechargeable lithium-ion battery**, which provides a compact and efficient power source for portability. To enable safe and convenient recharging, a **charging module** is connected to manage the charging cycle. A **step-down power module** is used to regulate voltage levels, ensuring that all components receive stable power without the risk of overvoltage. Additionally, an **electromagnetic buzzer** is used to provide audible alerts whenever the SOS system is triggered, making it easy to identify an active emergency even in noisy or outdoor environments. Together, these components support the core functions and enhance the usability and safety of the prototype.

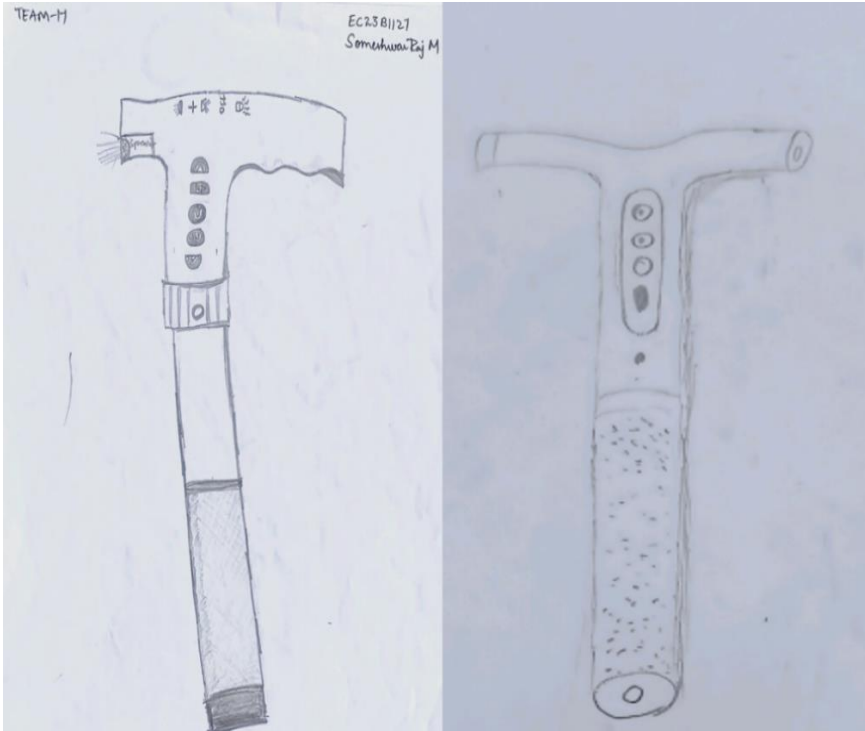
Components	QTY	Cost
38 pin CP2102 ESP32	1	492
MPU6050	1	298
NEO 6M GPS Module	1	233
Step Down (AMS1117-3.3 LDO 800MA)	1	25
C-Type Charging Module (TP4056)	1	28
Rechargeable Li-ion Battery (3.7v 2200mah)	1	270
Switch	1	10
Buzzer	1	23
Push Button	2	25
		1404

Sketches:

The initial phase of the design process involved brainstorming and rough sketching of how the components could be integrated directly into the walking stick itself. In these early concepts, we explored embedding the electronic modules—such as the microcontroller, sensors, and battery—within the body of the cane. While this approach seemed streamlined at first, it quickly became evident that it posed several challenges. Designing and fabricating a completely customized stick would require extensive mechanical work, reduce modularity, and limit the device's adaptability for different users or cane types.

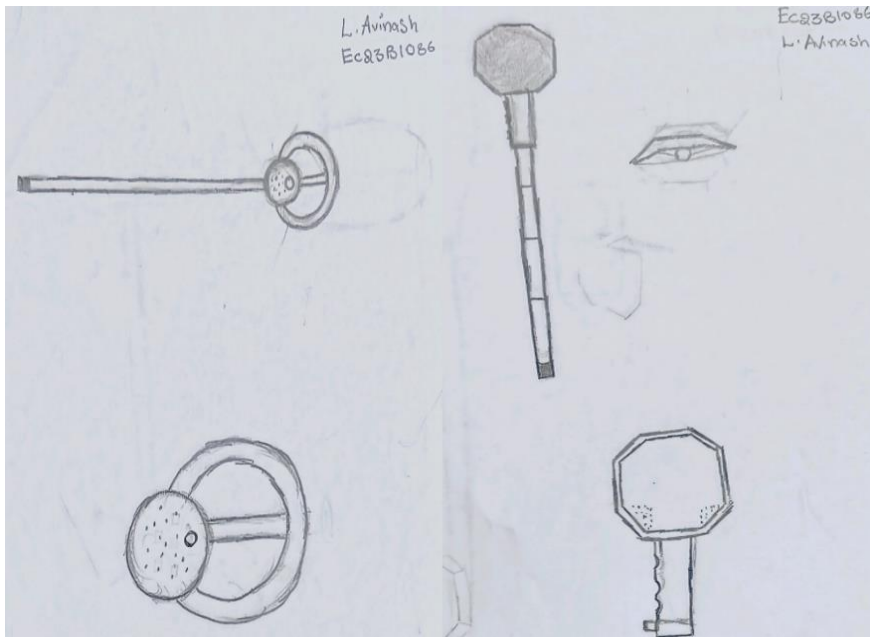
TEAM-M

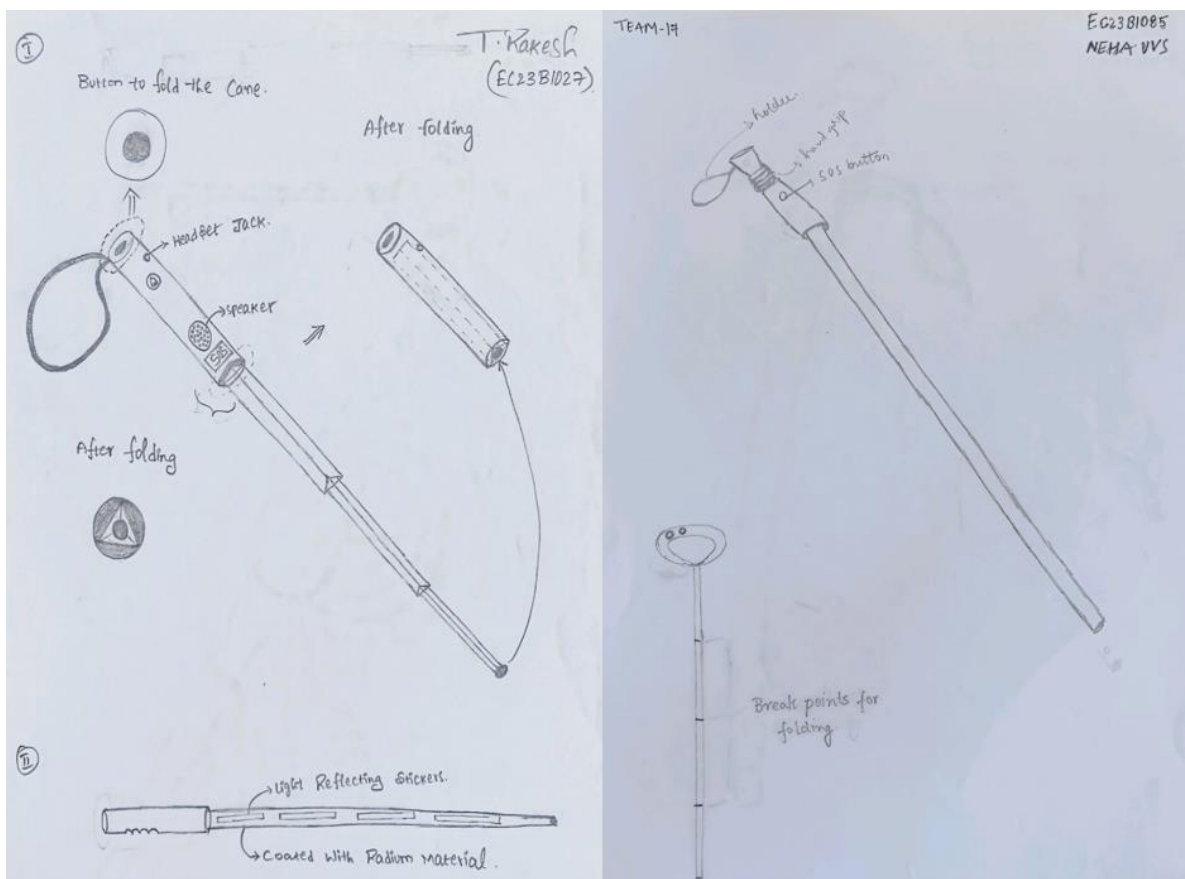
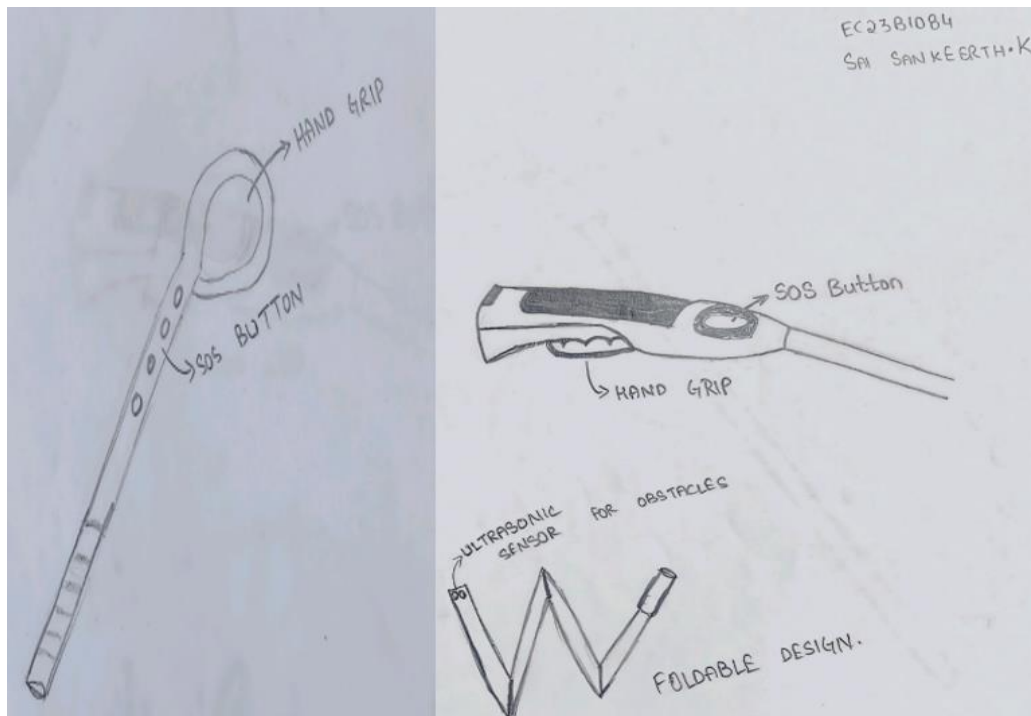
EC23B1127
Someshwar Baj M



L. Avinash
Ec83B1066

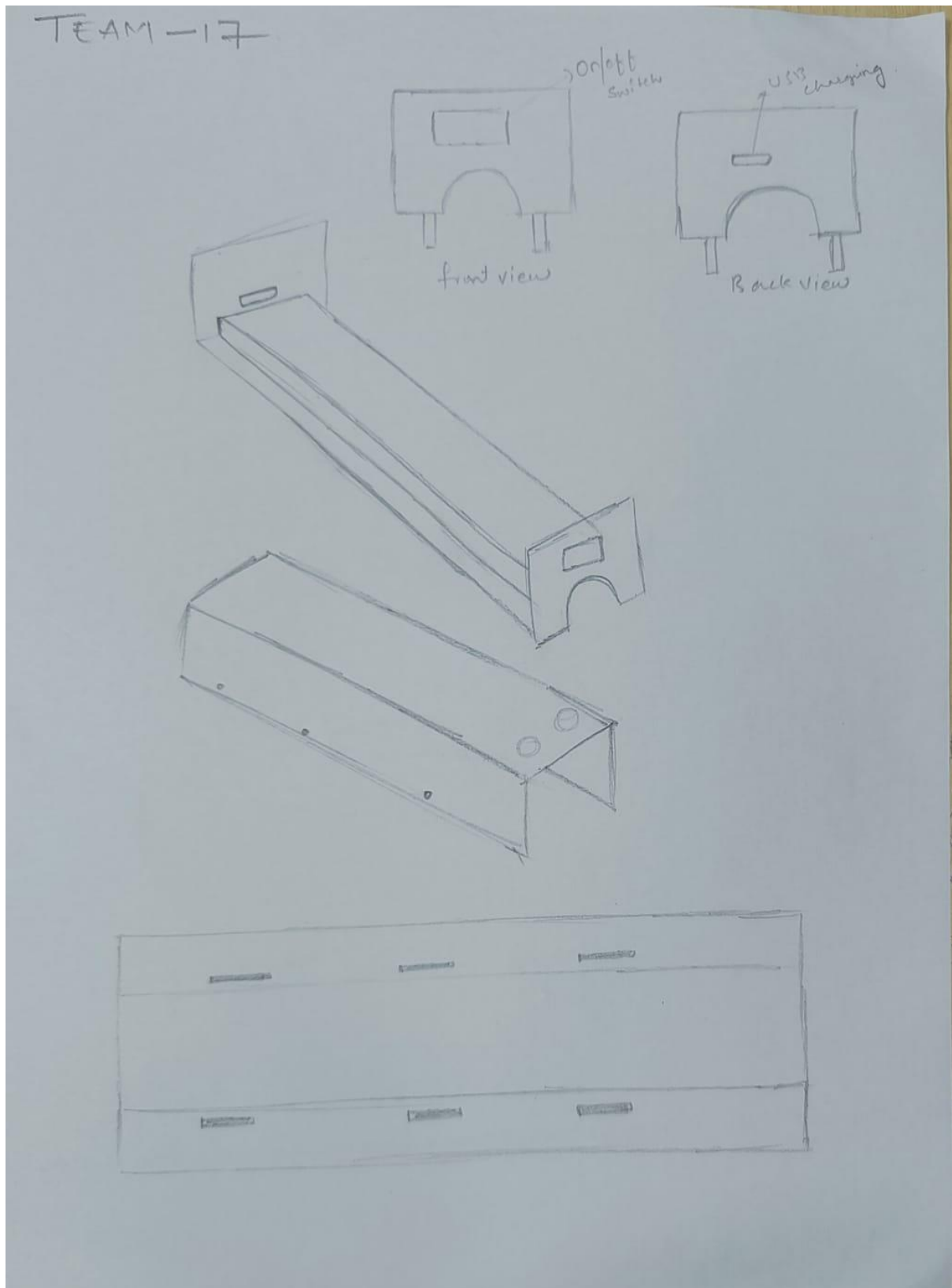
Ec83B1066
L. Avinash





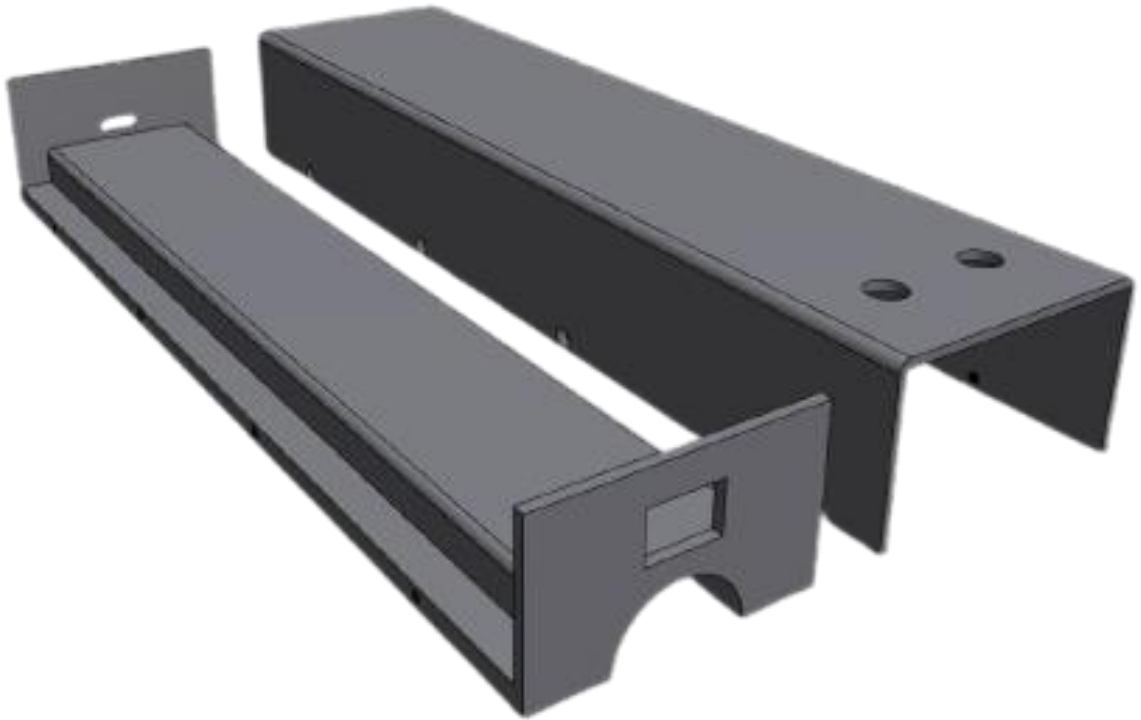
The refined concept of a modular, attachable device led to the development of the final prototype sketch, which illustrates the optimized layout and placement of all key components within a compact enclosure.

Final Sketch for the Prototype:



3D Model:

Building on the final sketch, a 3D model was designed and fabricated to bring the concept to life—real images of the printed prototype are shown below to demonstrate the physical structure and component arrangement.



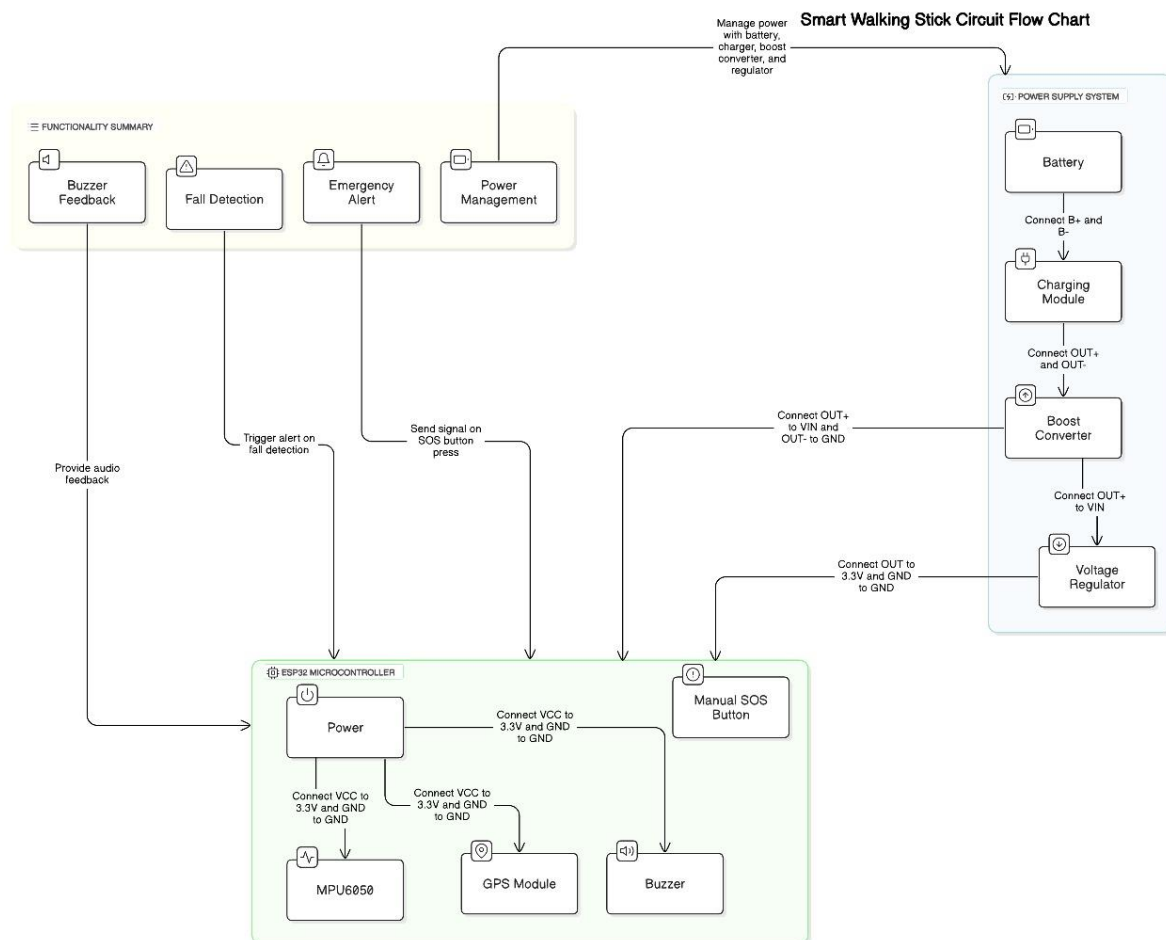


After finalizing the enclosure design, the 3D model was further refined to incorporate all essential components, ensuring proper alignment, accessibility, and compactness—shown below is the model with the internal layout tailored to fit each module efficiently.



Working Mechanism:

The circuit flow chart provides a structured overview of the working of the Opticane device. It is divided into three main sections: functionality summary, power supply system, and the ESP32 microcontroller connections. Each component is systematically mapped to show its role—ranging from fall detection and emergency alerts to power regulation. This diagram ensures a clear understanding of how all modules interact and how signals and power are distributed throughout the system.



Embedded System:

1. ESP32:

Role in System: The ESP32 microcontroller is the heart of the Opticane device. It reads data from the sensors, processes fall detection logic,

manages buttons and buzzer outputs, and sends emergency alerts over Bluetooth.

Code:

```
#include "BluetoothSerial.h"
BluetoothSerial SerialBT;
```

```
void setup() {
  Serial.begin(9600);
  SerialBT.begin("ESP32_Fall_Alert");
}
```

```
void loop() {
  SerialBT.println(sosData); // Sends SOS alert
  SerialBT.println(gpsData); // Sends fall alert
}
```

- Serial.begin(9600); – Initializes the serial monitor for debugging.
- BluetoothSerial SerialBT; and
- SerialBT.begin("ESP32_Fall_Alert"); – Initializes Bluetooth with a custom device name.
- pinMode(...) – Configures the buzzer and both buttons.
- SerialBT.println(...) – Sends emergency alerts or SOS messages to a paired phone via Bluetooth.

2. MPU6050:

Role in System: The MPU6050 is an accelerometer and gyroscope used to detect sudden movement or impact, which may indicate a fall.

Code:

```
#include <Wire.h>
#include <MPU6050.h>
```

```
MPU6050 mpu;
```

```

void setup() {
  Wire.begin();
  mpu.initialize();

  if (!mpu.testConnection()) {
    Serial.println(" ✗ MPU6050 connection failed!");
    while (1);
  }
  Serial.println(" ✓ MPU6050 connected!");
}

void loop() {
  int16_t ax, ay, az;
  mpu.getAcceleration(&ax, &ay, &az);
  long totalAcc = sqrt(ax * ax + ay * ay + az * az);

  if (totalAcc > IMPACT_THRESHOLD) {
    Serial.println(" ⚡ Sudden movement detected! ALERT!");
    // ... fall alert logic ...
  } }

```

- `mpu.initialize();` – Starts communication with the sensor.
- `mpu.getAcceleration(&ax, &ay, &az);` – Reads acceleration data in x, y, and z directions.
- `long totalAcc = sqrt(ax * ax + ay * ay + az * az);` – Calculates the total acceleration to detect sudden spikes.
- `if (totalAcc > IMPACT_THRESHOLD)` – If sudden motion is detected (greater than a defined threshold), it's considered a fall.

3. GPS Module:

Role in System: The GPS module is used to fetch real-time location coordinates when a fall or SOS event is triggered, so that help can be directed to the exact spot.

Code:

```
#include <TinyGPS++.h>
```



```
#include <HardwareSerial.h>
```

```
TinyGPSPlus gps;
```

```
HardwareSerial GPS_Serial(1); // Use UART1
```

```
void setup() {
```

```
    GPS_Serial.begin(9600, SERIAL_8N1, 16, 17); // TX=17, RX=16
```

```
}
```

```
void loop() {
```

```
    while (GPS_Serial.available()) {
```

```
        gps.encode(GPS_Serial.read());
```

```
    }
```

```
    if (gps.location.isValid()) {
```

```
        String gpsData = "Lat=" + String(gps.location.lat(), 6) +
```

```
                        ";Lon=" + String(gps.location.lng(), 6);
```

```
        Serial.println(gpsData);
```

```
    } else {
```

```
        Serial.println("Location Not Available");
```

```
    }
```

```
}
```

- `HardwareSerial GPS_Serial(1);` – Sets up UART1 to communicate with the GPS.
- `GPS_Serial.begin(9600, SERIAL_8N1, 16, 17);` – Starts GPS communication on GPIO 16 (RX) and 17 (TX).
- `gps.encode(GPS_Serial.read());` – Continuously feeds GPS data into the TinyGPS++ library for parsing.
- `gps.location.lat(), gps.location.lng()` – Retrieves current latitude and longitude to append in alerts.

Complete Code:

```
#include <Wire.h>
```

```
#include <MPU6050.h>
```

```
#include "BluetoothSerial.h"
```

```

#include <TinyGPS++.h>

#include <HardwareSerial.h>

MPU6050 mpu;

BluetoothSerial SerialBT;

TinyGPSPlus gps;

HardwareSerial GPS_Serial(1); // Use UART1 for GPS


const int buzzerPin = 26;    // Buzzer Pin
const int buttonPin = 12;    // Fall cancel button
const int sosButtonPin = 13; // Manual SOS button
const int IMPACT_THRESHOLD = 20000;


bool fallDetected = false;
unsigned long fallTime = 0;


void setup() {
    Serial.begin(9600);
    SerialBT.begin("ESP32_Fall_Alert");
    Wire.begin();
    mpu.initialize();
    GPS_Serial.begin(9600, SERIAL_8N1, 16, 17); // GPS on UART1


    pinMode(buttonPin, INPUT_PULLUP);
    pinMode(sosButtonPin, INPUT_PULLUP); // SOS button
    pinMode(buzzerPin, OUTPUT);


    if (!mpu.testConnection()) {
        Serial.println(" ✕ MPU6050 connection failed!");
    }
}

```

```

    while (1);
}

Serial.println("✅ MPU6050 connected!");
}

void loop() {
    // Check for manual SOS button press
    if (digitalRead(sosButtonPin) == LOW) {
        Serial.println("Manual SOS button pressed!");

        String sosData = "SOS ALERT! URGENT HELP REQUIRED!!:";
        if (gps.location.isValid()) {
            sosData += "Lat=" + String(gps.location.lat(), 6) + ";Lon=" + String(gps.location.lng(),
6);
        } else {
            sosData += "Location Not Available";
        }
        SerialBT.println(sosData);
        Serial.println(sosData);

        delay(2000); // Debounce delay to avoid multiple sends
    }

    // Read acceleration data
    int16_t ax, ay, az;
    mpu.getAcceleration(&ax, &ay, &az);
    long totalAcc = sqrt(ax * ax + ay * ay + az * az);

    Serial.print("Total Acc: ");

```

```
Serial.println(totalAcc);
```

```
// Read GPS Data
```

```
while (GPS_Serial.available()) {  
    gps.encode(GPS_Serial.read());  
}
```

```
if (totalAcc > IMPACT_THRESHOLD) {
```

```
    Serial.println("💣 Sudden movement detected! ALERT!");
```

```
    fallDetected = true;
```

```
    fallTime = millis();
```

```
    digitalWrite(buzzerPin, HIGH);
```

```
    Serial.println("🔊 Buzzer ON! Press button to cancel alert.");
```

```
    while (millis() - fallTime < 5000) {
```

```
        if (digitalRead(buttonPin) == LOW) {
```

```
            Serial.println("✅ Alert canceled by user.");
```

```
            fallDetected = false;
```

```
            digitalWrite(buzzerPin, LOW);
```

```
            return;
```

```
        }
```

```
        delay(500);
```

```
    }
```

```
    Serial.println("📶 No response! Sending Bluetooth alert...");
```

```
    String gpsData = "FALL DETECTED! NEED HELP!:";
```

```
    if (gps.location.isValid()) {
```

```

        gpsData += "Lat=" + String(gps.location.lat(), 6) + ";Lon=" + String(gps.location.lng(),
6);
    } else {
        gpsData += "Location Not Available";
    }

    SerialBT.println(gpsData);
    Serial.println(gpsData);

    fallDetected = false;
    digitalWrite(buzzerPin, LOW);
}
delay (500);}

```

The code is designed to monitor sudden movements and manual emergency triggers to ensure user safety. At the core of the system is the **ESP32**, which controls all components and manages Bluetooth communication. The **MPU6050** sensor continuously reads acceleration values in all three axes. These values are then used to compute the total force exerted on the sensor. If this value exceeds a defined threshold (`IMPACT_THRESHOLD = 20000`), it is interpreted as a fall.

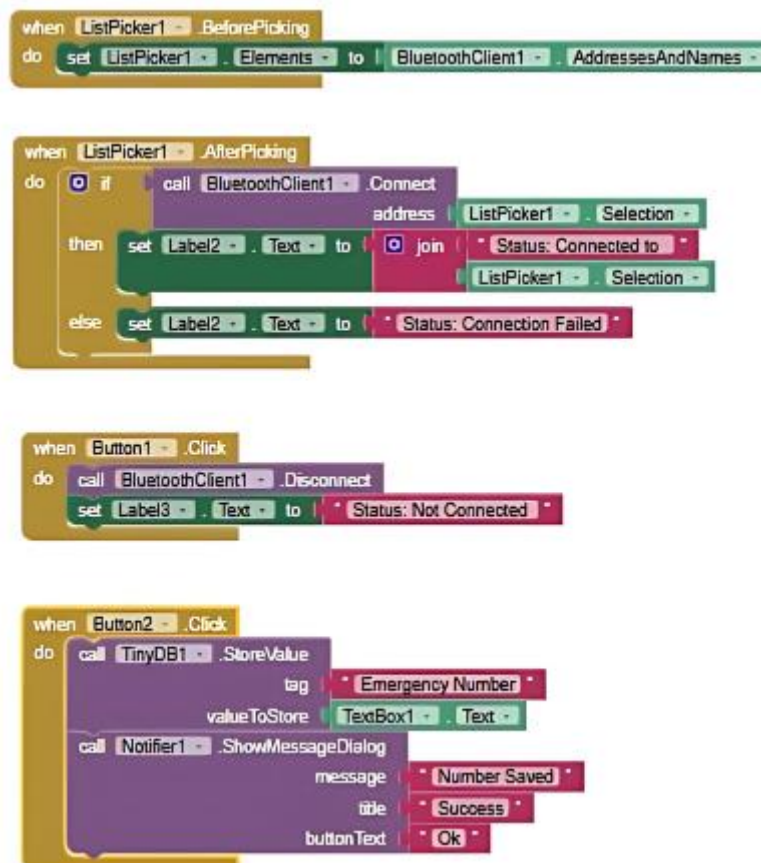
Upon fall detection, the **buzzer** is activated immediately to alert the surrounding people. The buzzer remains **ON for 5 seconds**, giving the user an opportunity to cancel the alert by pressing a designated **cancel button** (button Pin). If the user does not respond within this window, the system assumes a genuine emergency and automatically sends a **Bluetooth alert message** to a connected mobile device. This message contains real-time **GPS coordinates** if available, helping caregivers or contacts locate the user.

Additionally, a **manual SOS button** is provided, which when pressed, sends an instant emergency alert regardless of fall detection. This ensures that users can seek help in scenarios where a fall might not occur but assistance is still needed. Overall, the system prioritizes both **automated detection** and **manual intervention**, providing a reliable safety mechanism with timely alerts and geolocation data.

App Integration:

The wireless communication system of OptiCane is facilitated through a custom-built mobile application, designed using **MIT App Inventor's** block-based programming interface. This user-friendly app enables seamless Bluetooth connectivity with the ESP32 microcontroller embedded in the walking stick. The app not only allows the user to connect or disconnect from the device with ease but also provides real-time feedback on connection status. Additionally, it offers functionality to store an emergency contact number, which is used in critical alerts triggered by the device, ensuring quick communication during emergencies.

1. Bluetooth & Emergency Setup:



The image above shows the block-based logic used in the MIT App Inventor for managing Bluetooth connections and saving emergency contact numbers for the smart walking stick app interface.

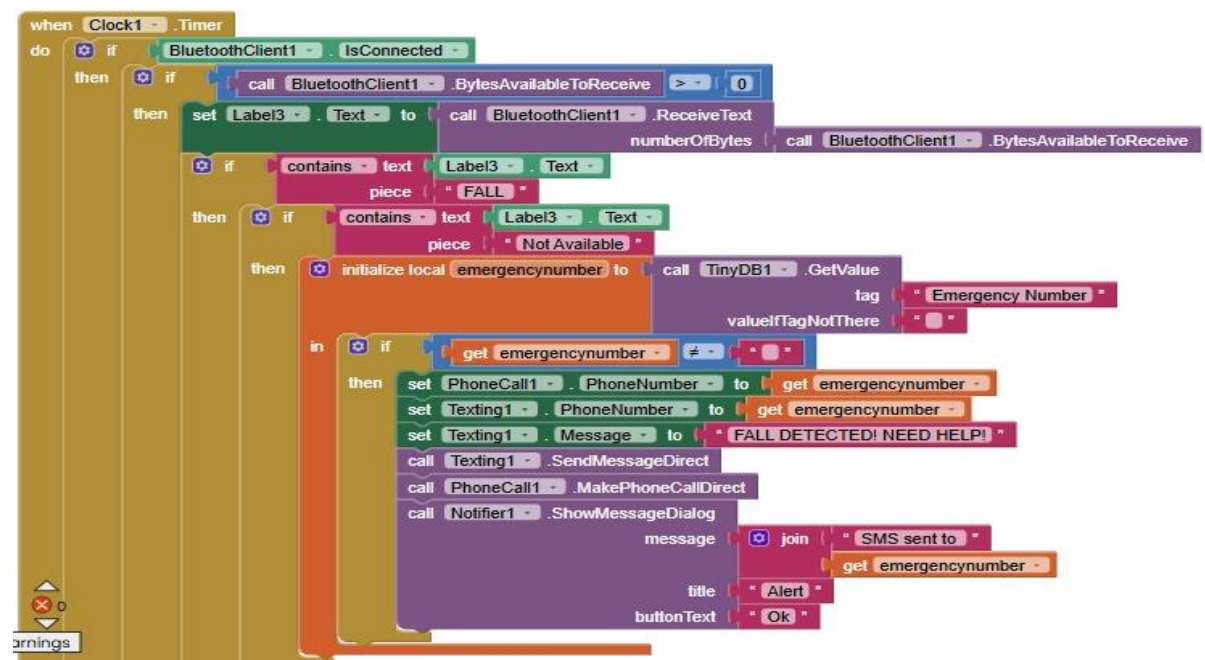
1. Bluetooth Connection Handling

- **ListPicker1.BeforePicking:**
When the user taps the list to connect to a Bluetooth device, this block first populates the list with available nearby Bluetooth devices using `BluetoothClient1.AddressesAndNames`.
- **ListPicker1.AfterPicking:**
Once the user selects a device from the list:
 - The app attempts to connect to the selected device using `BluetoothClient1.Connect`.
 - If successful, it updates **Label2** with the message Status: Connected to [Device Name].
 - If the connection fails, it sets **Label2** to display Status: Connection Failed.
- **Button1.Click (Disconnect):**
This button allows the user to disconnect from the currently connected device.
 - On clicking, it calls `BluetoothClient1.Disconnect` and updates **Label3** to Status: Not Connected.

2. Emergency Number Storage

- **Button2.Click (Save Number):**
This button enables users to save an emergency contact number locally on the device.
 - It stores the number entered in **TextBox1** under the tag "Emergency Number" using `TinyDB1.StoreValue`.
 - After saving, it shows a confirmation dialog with the message: "**Number Saved**", titled "**Success**", and a button labelled "**OK**" using the `Notifier1` component.

2. Fall Detection via Bluetooth:



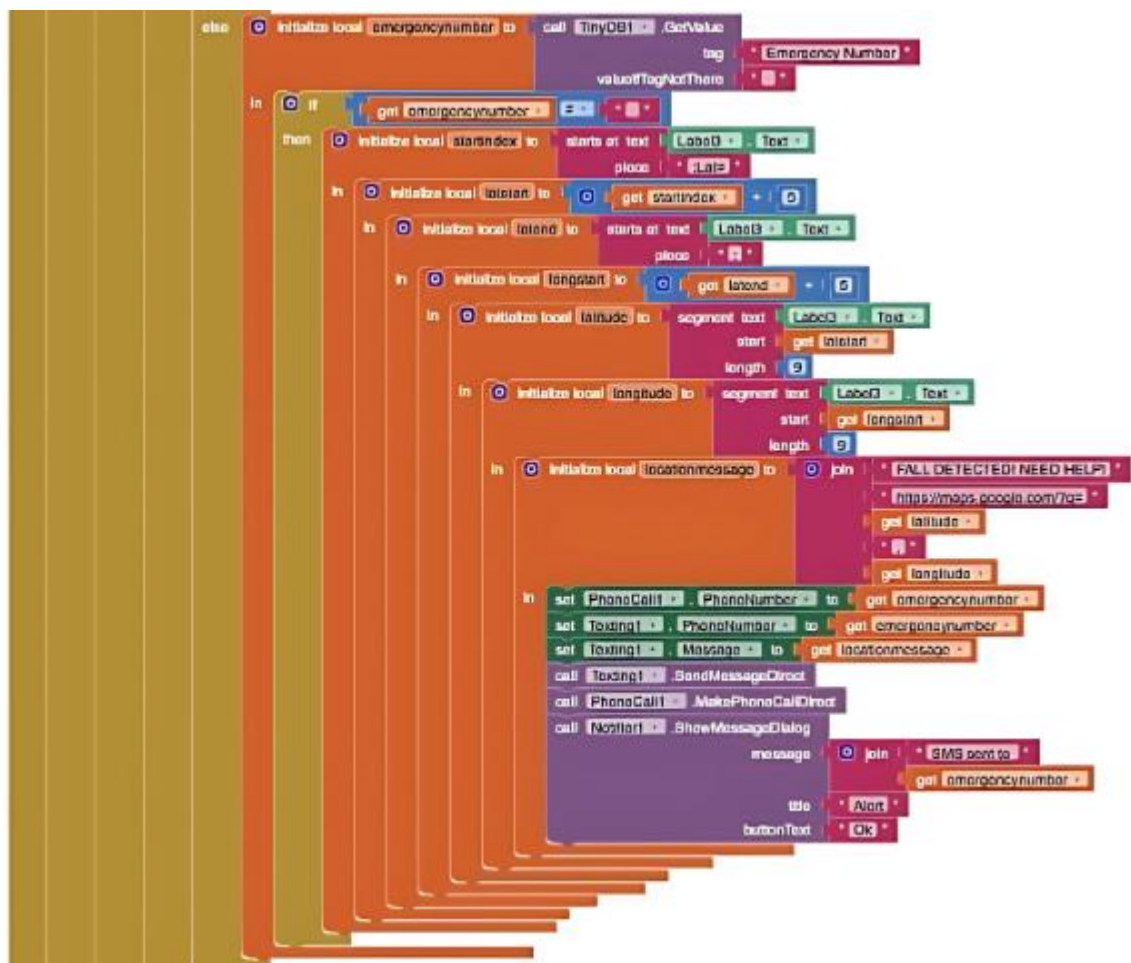
The above image shows the App Inventor block logic used to handle fall detection alerts via Bluetooth communication. Here's a breakdown of the functionality:

1. **Clock Timer Trigger:** The logic runs periodically when the Clock1.Timer event triggers. It first checks whether the Bluetooth client (BluetoothClient1) is connected.
2. **Receiving Data:** If connected, it checks if there are bytes available to receive. If yes, it reads the incoming message using .Receive Text and stores it in Label3.
3. **Fall Detection Check:** The received text is then checked for the keyword "**FALL**". If the message does not include "Not Available" (used as a default/failure value), the fall alert process is initiated.
4. **Emergency Number Retrieval:** The emergency contact number is fetched from the local database (TinyDB1) using the tag "Emergency Number".
5. **Alert Actions:**
 - The number is assigned to both the **Phone Call** and **Texting** components.
 - An SMS with the message "**FALL DETECTED! NEED HELP!**" is sent to the emergency number.

- A phone call is immediately triggered to the same number.
- A message dialog (Notifier) is displayed confirming that the SMS was sent, mentioning the emergency number.

This block logic ensures an automated and immediate response to a detected fall, allowing the system to alert caregivers through both SMS and direct call—making it a critical part of the safety functionality in the OptiCane project.

3. Fall Detection with Location:



This image showcases an extended MIT App Inventor block logic that builds on the basic fall detection alert system by including real-time location sharing via Google Maps in the emergency message.

Key Functionalities:

1. **Emergency Number Fetching:**

The logic starts by retrieving the emergency contact from the local database (TinyDB1) using the tag "Emergency Number".

2. **Location Extraction from Bluetooth Data:**

- The message received via Bluetooth (Label3.Text) includes GPS coordinates along with the "FALL" alert.
- The block extracts latitude and longitude by segmenting the string based on known delimiters.
- These segments are then used to form a **Google Maps link** to pinpoint the user's location.

3. **Creating the Emergency Message:**

The final message includes:

php-template

CopyEdit

FALL DETECTED! NEED HELP!

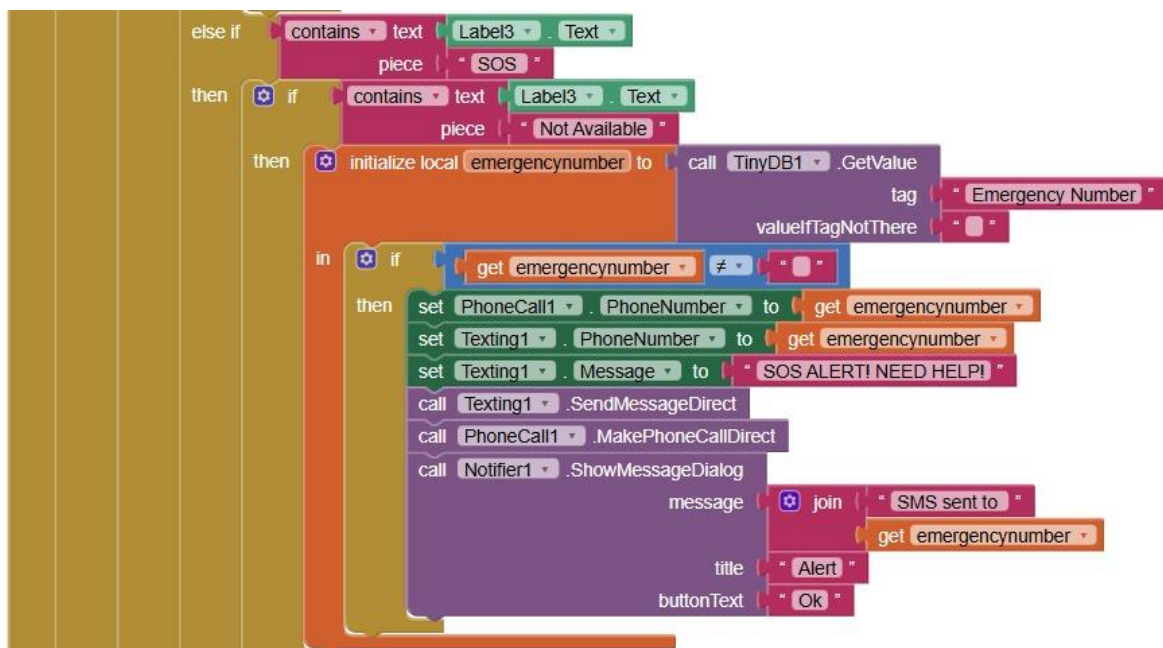
Location: <https://maps.google.com/?q=<latitude>,<longitude>>

This message is stored in a local variable named locationmessage.

4. **Sending Alert:**

- The **Texting** component sends the generated message to the saved emergency number.
- The **PhoneCall** component initiates a direct call to the same number.
- A **Notifier** displays a dialog confirming the SMS has been sent, showing the number it was sent to.

4. SOS Alert via Bluetooth:



This block diagram from MIT App Inventor represents the logic for detecting a manual SOS trigger sent via Bluetooth and responding with an emergency alert.

Key Functionalities:

1. SOS Keyword Detection:

- The block checks if the received Bluetooth message (Label3.Text) contains the keyword "SOS".
- It also verifies that the data is **not labelled as "Not Available"**, ensuring it's valid for further action.

2. Emergency Contact Retrieval:

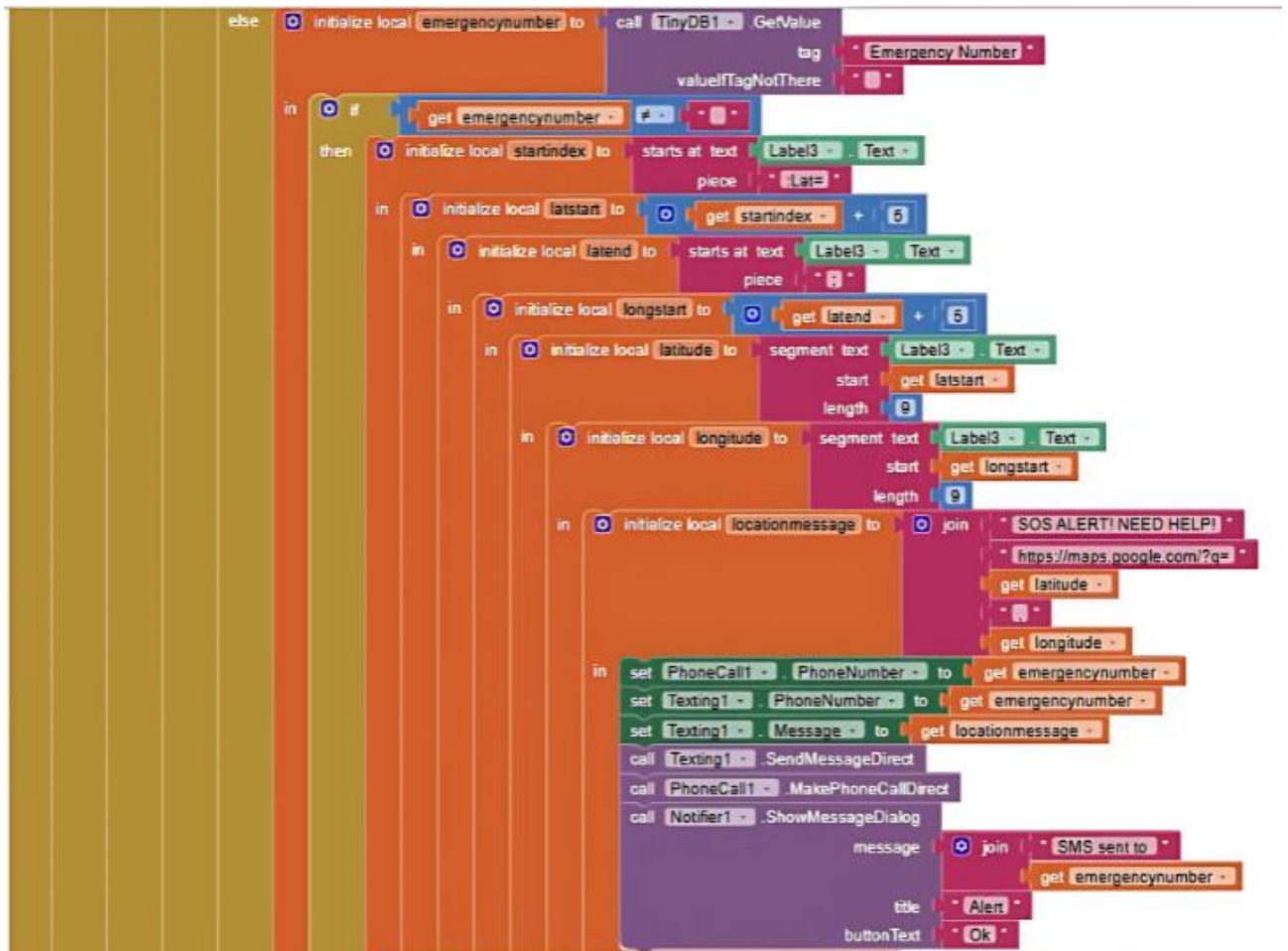
- The app retrieves the stored emergency number from TinyDB1 using the tag "Emergency Number".
- If the number is successfully fetched (i.e., not empty), the system proceeds to initiate alerts.

3. Alert Actions Triggered:

- The app sends a **text message** saying "SOS ALERT! NEED HELP!" to the emergency number.

- Simultaneously, a **direct phone call** is placed to the same contact using PhoneCall1.
- A **notification popup** confirms the message has been sent and shows the emergency number.

5. SOS Alert with Location:



Based on the uploaded image, here's a breakdown of the logic and flow used in the MIT App Inventor block-based code (referencing the shared thread):

Purpose:

This block of code is part of an emergency alert system. When triggered, it:

1. Extracts GPS coordinates (latitude and longitude).
2. Formats a Google Maps link with the location.

3. Sends an SOS message with the location via SMS.
4. Makes an emergency phone call.
5. Displays a notification.

Detailed Flow:

1. Emergency Number Retrieval:

plaintext

CopyEdit

emergencyNumber ← TinyDB1.GetValue("Emergency Number")

2. If emergency number is not empty ("0"):

- The code continues.

3. Extract Location from Label3.Text:

- Parses the text in Label3.Text to find location data using specific indexes.
- startIndex, startLat, latEnd, longStart, etc., are calculated to extract:
 - latitude
 - longitude

4. Compose Location Message:

plaintext

CopyEdit

locationMessage ← "SOS ALERT! NEED HELP!\nhttps://maps.google.com/?q="+ latitude + "," + longitude

5. Set up Texting and Calling:

- Set phone number for texting and calling to emergencyNumber
- Message set to locationMessage

- SMS is sent using `Texting1.SendMessageDirect`
- Phone call initiated using `PhoneCall1.MakePhoneCallDirect`
- Notification displayed with:

plaintext

CopyEdit

"SMS sent to " + emergencyNumber

UI Feedback:

- Alert message shown with the `Notifier1.ShowMessageDialog`.

Overview of Opticane App:

Purpose:

This app is designed to **automatically alert emergency contacts** with your live GPS location in case of distress. It performs three main actions:

1. **Sends an SOS text** with your real-time location.
2. **Makes a direct phone call** to your emergency number.
3. **Displays a confirmation notification** to the user.

How It Works (Flow Overview):

1. Emergency Number Retrieval:

- The app fetches a stored emergency contact number from TinyDB.
- If the number is "0" or not set, it doesn't proceed.

2. Location Parsing from Label:

- It extracts latitude and longitude values embedded in `Label3.Text` (assuming it contains location in a formatted string like: "Lat:12.3456, Long:78.9012").
- Parsing is done using string manipulation blocks (starts at, segment, etc.).

3. Message Composition:

- Combines the extracted coordinates into a clickable Google Maps link.
- Creates a message:

php-template

CopyEdit

SOS ALERT! NEED HELP!

<https://maps.google.com/?q=<latitude>,<longitude>>

4. Communication Actions:

- **SMS:** Sends the above message using `Texting1.SendMessageDirect`.
- **Call:** Initiates a direct call to the emergency contact using `PhoneCall1.MakePhoneCallDirect`.

5. User Feedback:

- Shows a popup using `Notifier1.ShowMessageDialog` that confirms:

css

CopyEdit

SMS sent to <Emergency Number>

Highlights:

- Real-time **location sharing** in emergencies.
- Dual-mode alert system: **SMS + Phone Call**.
- **Offline capability:** SMS and calls don't need internet.
- Uses **TinyDB** for persistent storage of emergency contact.

App Interface:



The image above shows the **user interface (UI)** of the mobile application titled **"Fall Detection Alert System"**, developed using **MIT App Inventor**. This app serves as the control center for the entire emergency system, enabling Bluetooth communication with the hardware and handling alert functionalities.

Interface Elements and Their Functions:

1. Title Bar:

- Displays the name of the app and screen title: **"Fall detection Alert System"**.

2. Bluetooth Controls:

- **Select Bluetooth Device:** Initiates device discovery, allowing the user to connect with the ESP32 hardware via Bluetooth.
- **Disconnect Bluetooth:** Terminates the Bluetooth connection when it's no longer needed.

3. Connection Status:

- A **label** shows the current Bluetooth connection status, e.g., **"Status: Not Connected"**. It dynamically updates based on the connection state.

4. Emergency Contact Input:

- A **text box** is provided for the user to input their emergency contact number.

- **Save Number:** Stores the entered contact number into the app's local database (TinyDB), used later when sending alerts.

5. Received Alert Section:

- A **label** or text field is designated to display real-time messages or alerts received from the hardware, such as:
 - "Fall detected"
 - "SOS activated"
 - "Location sent"

Circuit Diagram:

The hardware architecture of the SOS Emergency Alert System. It integrates various sensors and modules with the ESP32 microcontroller to detect emergencies and trigger alerts via buttons. Below is a breakdown of the connected components and their functions:

Power Supply

- A 3.7V Li-ion battery is used as the primary power source.
- Connected through a TP4056 charging module which allows USB charging and provides regulated output to the circuit.
- A toggle switch is added to turn the system ON or OFF.

GPS Module (NEO-6M)

- Used to obtain the real-time geographic coordinates (latitude and longitude) of the user.
- Connected to the ESP32 for serial communication.
- Crucial for generating the Google Maps location link in emergency alerts.

ESP32 Microcontroller

- The central processing unit that controls all sensors, buttons, and communication.
- It reads sensor data, detects button presses, processes GPS data, and sends alerts through mobile connectivity or Bluetooth (if programmed).

SOS & Help Buttons

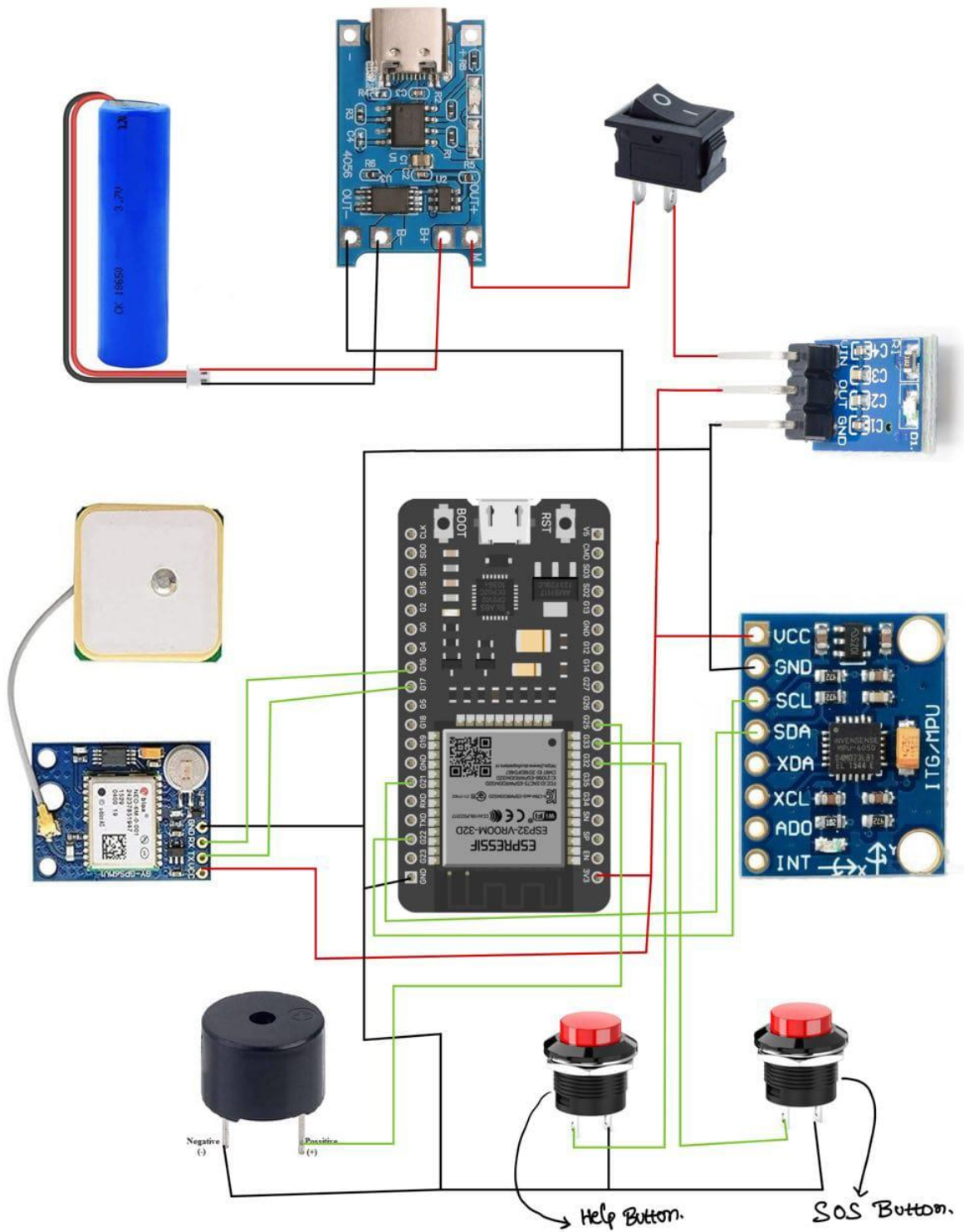
- Two push buttons:
 - SOS Button: Triggers immediate alert with location.
 - Help Button: Can be programmed for non-emergency notifications or secondary alert mode.
- Both buttons are connected to digital input pins of the ESP32.

Buzzer

- An active buzzer provides audible feedback when an emergency alert is triggered.
- Acts as a local alert to indicate that the system has activated.

MPU6050 (Accelerometer + Gyroscope)

- Monitors motion and orientation.
- Can be used to detect falls, shakes, or sudden movement, potentially triggering automatic alerts if abnormal activity is detected.



PROTOTYPE

This is our final prototype

CAD MODEL



Real model



Conclusion:

Opticane is a smart, portable emergency alert system designed for individuals who may be at risk of falling, getting lost, or needing urgent help—such as the elderly, people with disabilities, women, or solo travellers. The system integrates real-time fall detection, SOS alerting, and GPS-based location sharing using a combination of hardware components and a mobile app.

Opticane is easy to use smart emergency response system. Whether for senior citizens prone to falling, individuals in remote areas, or anyone needing a discreet safety device, Opticane offers reliability, mobility, and peace of mind. With both automated and manual triggers, and dual alert modes the system ensures help is always just a press or a fall away.

Future Enhancements:

1. Battery Monitoring System:

- Display real-time battery percentage on the app.
- Send a low-battery warning SMS to the emergency contact.

2. SIM Module Integration:

- Add a SIM GSM module to make the device fully independent (no need for Bluetooth or mobile app to send SMS/Call).
- Useful in situations where the phone isn't available or connected.

3. IoT Cloud Integration:

- Push alerts and live location to cloud dashboards like Firebase or Blynk.
- Allow family/caregivers to monitor the user remotely from any device.

4. AI-Powered Fall Detection:

- Use machine learning algorithms to improve fall detection accuracy.
- Reduce false positives from normal movements like sitting or bending.

5. Voice Feedback System:

- Add a voice module to guide or alert the user during operations (e.g., "Help alert sent", "Battery low", etc.).

6. Location Tracking History:

- Store and view historical movement data of the user over time.
- Useful for medical and care records.

7. Two-Way Communication:

- Enable response SMS or call back from the emergency contact to the device.
- Useful for reassurance or communication after an alert.

8. Compass & Navigation Aid:

- Add basic navigation features using onboard sensors to guide visually impaired users (e.g., to the nearest hospital).

9. Environmental Sensors:

- Include temperature, gas leak, or humidity sensors for additional health and safety monitoring.

10. Cross-Platform App:

- Develop a polished version of the app for both Android and iOS with improved UI and user experience.

REFERENCES

- Rashid, O., Johnson, C., & Yongchareon, S. (2017). Smart Walking Stick for Elderly Based on IoT System. International Journal of Engineering & Technology (UAE), 7(2.29), 1-4.
- Zhang, Z., Wang, L., & Wang, X. (2019). An IoT-Based Smart Cane for Elderly People Fall Detection Using Deep Learning. IEEE Access, 7, 49610-49619.